

**Database Management Systems
(UCS310)**

Lab Project Report

Agriculture Management System

Submitted By:

Prabhjot Singh (1024030978)

Akashdeep Singh(1024030983)

Pahuldeep Singh(1024030982)

Submitted To:

Ms.Abhishelly Sharma



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

B.Tech (2nd Year)

DEPARTMENT OF COMPUTER ENGINEERING

**THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY,
(A DEEMED TO BE UNIVERSITY), PATIALA, PUNJAB, INDIA**

2. Introduction

Agriculture is the backbone of India's economy, employing over 50% of the workforce and contributing significantly to national food security. Yet, farm-level management — tracking soil health, applying fertilizers correctly, monitoring crop diseases, and maintaining records of field activities — remains largely manual and fragmented across most small and medium-scale farms.

The Agriculture Management System is a full-stack web application that brings structured, database-driven intelligence to farm operations. It enables farmers and agronomists to manage multiple fields with complete soil nutrient profiles (Nitrogen, Phosphorus, Potassium, pH), log crop disease incidents with severity classification, track every fertilizer application, and receive automated alerts when critical situations arise — all through a centralized relational database.

Motivation

Manual record-keeping in agriculture leads to data loss, inconsistency, and poor decision-making. Farmers often cannot correlate soil pH trends with crop yield, or track cumulative fertilizer usage per field season. A relational database system provides the structure, constraints, and automation required to address these gaps systematically.

Why a DBMS Instead of File-Based System

- File-based systems have no referential integrity — a deleted field record leaves orphan soil/fertilizer entries.
- DBMS enforces ACID properties, ensuring every transaction is atomic and consistent.
- SQL allows complex multi-table queries (joins, aggregations) that are impossible in flat files.
- Triggers and stored procedures automate disease alerting and audit logging without application-level code.
- Concurrent multi-user access is managed by the DBMS with locking and isolation, unlike flat files.

3. Problem Statement

Indian agriculture suffers from an acute lack of structured data management at the field level. Farmers typically maintain paper-based or spreadsheet records for soil tests, fertilizer purchases, and disease observations — these records are siloed, error-prone, and impossible to query analytically.

Limitations of the Existing / Manual System

- No relational linkage between fields, soil data, fertilizer logs, and disease incidents.
- Duplicate and inconsistent records due to manual entry across multiple files or registers.
- No automated alerting when critical crop disease severity is detected.
- Inability to generate aggregated reports such as total fertilizer applied per season or fields with suboptimal pH.
- No immutable audit trail for fertilizer applications — making accountability impossible.
- Seasonal data loss due to physical register damage or misplacement.

Scope of Database Usage

A relational database system addresses all the above limitations by:

- Structuring all farm data into normalized, interrelated tables.
- Enforcing data integrity via primary keys, foreign keys, and check constraints.
- Automating critical workflows (disease alerts, audit logging) through triggers.
- Providing a stored procedure interface for fast, aggregated field summaries.
- Using a MySQL Event Scheduler for background recalculation of global fertilizer statistics.

4. Objectives of the Project

1. To design a normalized database using E–R data modeling covering all agricultural domain entities: Farmers, Fields, SoilRecords, FertilizerUsage, CropDiseases, Alerts, AuditLog, and FertilizerSummary.
2. To convert the ER model into a fully relational schema with correct primary keys, foreign keys, and referential integrity constraints.
3. To apply normalization up to the Fifth Normal Form (5NF), eliminating all partial, transitive, and multi-valued dependencies.
4. To implement the database using SQL DDL and DML — including table creation, data insertion, complex SELECT queries with joins, subqueries, aggregate functions, GROUP BY, HAVING, and views.
5. To implement PL/SQL-style constructs including: stored procedures for field summaries, functions for utility calculations, triggers for automated alerting and audit logging, cursors for row-by-row processing, and exception handling for data validation.
6. To ensure data consistency using constraints, transactions (COMMIT, ROLLBACK, SAVEPOINT), and ACID property enforcement.

5. Scope of the Project

Functional Boundaries

- Registration and management of farmer profiles and their associated agricultural fields.
- Periodic soil health recording with N, P, K nutrient levels and pH measurements per field.
- Fertilizer application tracking with quantity validation and automatic audit logging.
- Crop disease logging with severity classification and automated alert generation for HIGH/CRITICAL cases.
- Background recalculation of global fertilizer statistics via scheduled events.
- Field summary retrieval through a stored procedure returning aggregated soil, fertilizer, and disease data.

Types of Users Involved

User Role	Responsibilities
Agronomist / Admin	Manages farmer records, field data, soil reports, and generates field summaries.
Field Supervisor	Logs fertilizer applications, records crop disease incidents.
System (Automated)	Trigger-based alert generation, audit logging, event scheduler for summary updates.

Modules Covered

- Farmer & Field Management Module
- Soil Health Monitoring Module
- Fertilizer Usage Tracking Module (with trigger-based audit logging)
- Crop Disease & Alert Module (with trigger-based auto-alerting)
- Analytics & Reporting Module (stored procedure + event scheduler)

6. Proposed System Description

The Agriculture Management System is a premium full-stack web application with a React + Vite frontend and a Node.js/Express backend, powered entirely by a MySQL 8.0 relational database. The system design philosophy is database-first — all critical business logic (validation, alerting, auditing, aggregation) lives inside the database engine rather than the application layer.

Overall Working

- The frontend provides interactive dashboards with animated charts comparing N/P/K values across fields, live alerts panels, and activity log viewers.
- The backend exposes REST API endpoints that invoke SQL queries and stored procedures on the MySQL database.
- Farmers and fields are registered through form-based interfaces. Each field is linked to exactly one farmer via a foreign key constraint.
- Soil readings are periodically inserted per field. The dashboard renders grouped bar charts for nutrient comparison.
- Fertilizer applications are inserted with automatic validation (quantity capped at 100 kg by trigger) and audit trail generation.
- Disease incidents are logged with severity. HIGH and CRITICAL entries trigger an automatic alert insertion.

Key Features

- 3 database triggers: data validation, automated disease alerting, fertilizer audit logging.
- 1 stored procedure: GetFieldSummary — returns complete field analytics in a single call.
- 1 MySQL event: RecalculateFertilizerSummary — runs every 24 hours to refresh global stats.
- 1 stored function: GetTotalFertilizerForField — returns total kg applied to a given field.
- Full normalization to 5NF across 8 tables.

How the System Improves Efficiency

- Eliminates manual record duplication — all data stored once in normalized tables.
- Reduces response time for field analytics via stored procedure caching.
- Ensures zero-miss alerting for critical diseases — triggers fire at the database layer regardless of application state.

7. Database Design

7.1 Entity–Relationship (ER) Diagram

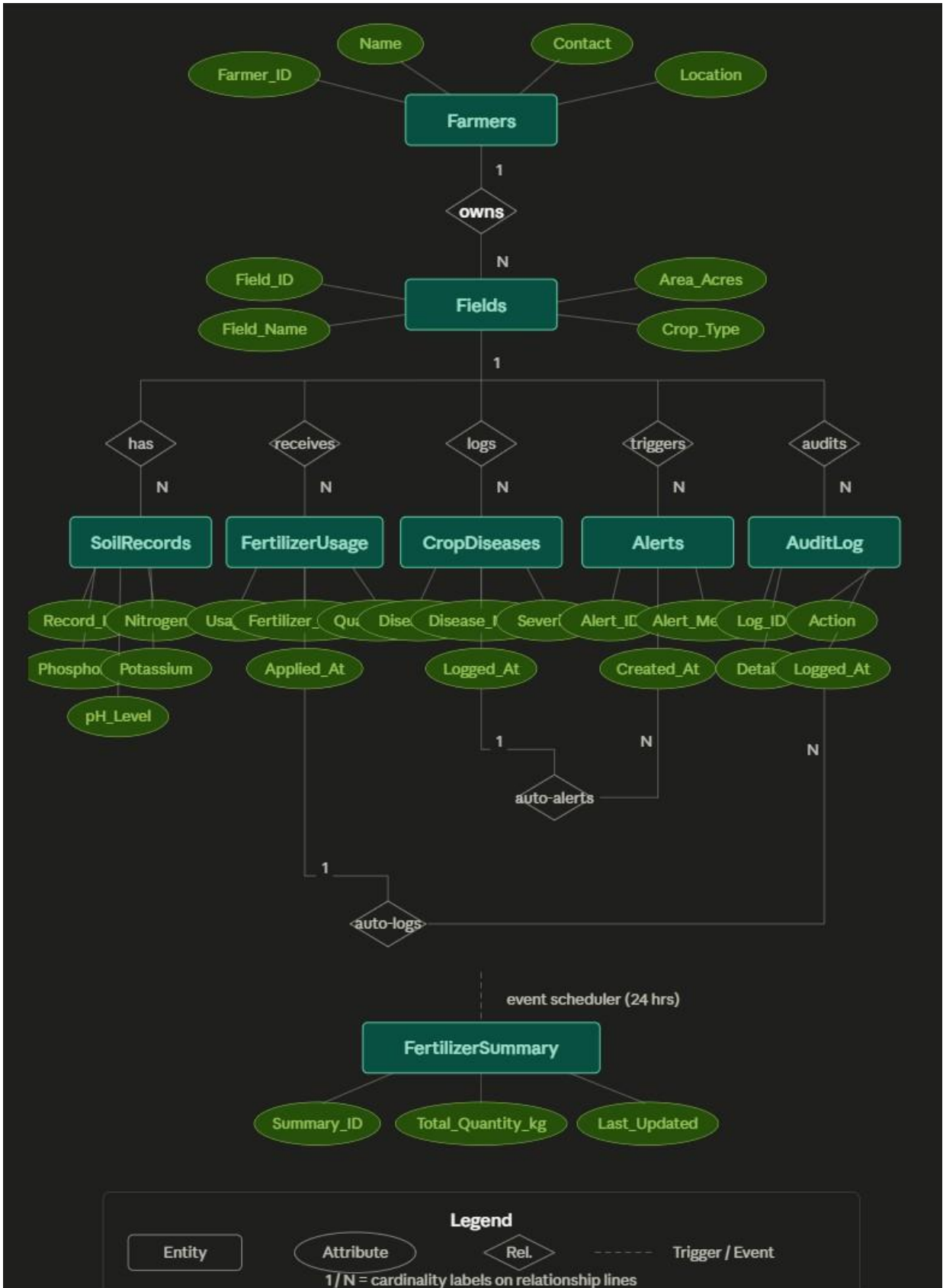
The ER diagram follows Chen notation: rectangles represent entities, ellipses represent attributes, and diamonds represent relationships.

Entities Identified:

- Farmers — stores farmer profile data.
- Fields — stores individual agricultural plot data, linked to Farmers.
- SoilRecords — stores periodic soil nutrient readings per field.
- FertilizerUsage — records each fertilizer application event per field.
- CropDiseases — logs disease incidents per field with severity.
- Alerts — auto-populated by trigger on HIGH/CRITICAL disease entries.
- AuditLog — immutable log auto-populated by trigger on fertilizer inserts.
- FertilizerSummary — cached aggregate updated by MySQL Event Scheduler.

Relationships:

- Farmers owns Fields (1:N) — one farmer can own many fields.
- Fields has SoilRecords (1:N) — one field has many soil readings over time.
- Fields receives FertilizerUsage (1:N) — one field has many fertilizer applications.
- Fields logs CropDiseases (1:N) — one field can have many disease incidents.
- Fields triggers Alerts (1:N) — one field can have many alerts.
- Fields audits AuditLog (1:N) — one field has many audit entries.
- CropDiseases auto-alerts Alerts (1:N) — via HighSeverityDiseaseAlert trigger.
- FertilizerUsage auto-logs AuditLog (1:N) — via LogFertilizerUsage trigger.



7.2 Relational Schema

Conversion of ER diagram into normalized relational tables:

Table	Columns	Key Constraints
Farmers	Farmer_ID, Name, Contact, Location	PK: Farmer_ID
Fields	Field_ID, Field_Name, Area_Acres, Crop_Type, Farmer_ID	PK: Field_IDFK: Farmer_ID
SoilRecords	Record_ID, Field_ID, Nitrogen, Phosphorus, Potassium, pH_Level, Recorded_At	PK: Record_IDFK: Field_ID
FertilizerUsage	Usage_ID, Field_ID, Fertilizer_Name, Quantity_kg, Applied_At	PK: Usage_IDFK: Field_ID
CropDiseases	Disease_ID, Field_ID, Disease_Name, Severity, Logged_At	PK: Disease_IDFK: Field_ID
Alerts	Alert_ID, Field_ID, Alert_Message, Created_At	PK: Alert_IDFK: Field_ID
AuditLog	Log_ID, Field_ID, Action, Details, Logged_At	PK: Log_IDFK: Field_ID
FertilizerSummary	Summary_ID, Total_Quantity_kg, Last_Updated	PK: Summary_ID

8. Normalization

Functional Dependencies Identified

- Farmer_ID → Name, Contact, Location
- Field_ID → Field_Name, Area_Acres, Crop_Type, Farmer_ID
- Record_ID → Field_ID, Nitrogen, Phosphorus, Potassium, pH_Level, Recorded_At
- Usage_ID → Field_ID, Fertilizer_Name, Quantity_kg, Applied_At
- Disease_ID → Field_ID, Disease_Name, Severity, Logged_At
- Alert_ID → Field_ID, Alert_Message, Created_At
- Log_ID → Field_ID, Action, Details, Logged_At

Normalization Process

1NF (First Normal Form):

All tables satisfy 1NF — every attribute holds atomic values and each table has a unique primary key. No multi-valued or composite attributes exist.

2NF (Second Normal Form):

All tables satisfy 2NF — since every primary key is a single-attribute surrogate key (e.g., Farmer_ID, Field_ID), there are no composite keys, and therefore no partial dependencies are possible.

3NF (Third Normal Form):

All tables satisfy 3NF — there are no transitive dependencies. In Fields, Field_Name, Area_Acres, and Crop_Type depend only on Field_ID, not on Farmer_ID or any other non-key attribute.

BCNF (Boyce-Codd Normal Form):

All functional dependencies have the primary key as their determinant. No overlapping candidate keys exist. The database satisfies BCNF.

4NF (Fourth Normal Form):

No multi-valued dependencies exist. A field has one crop type per record; soil nutrients are stored per-record as independent atomic values. The database satisfies 4NF.

5NF (Fifth Normal Form):

All tables are fully decomposed without loss of information. The Bookings pattern (many-to-many resolution) is not required here as every relationship is 1:N. All join dependencies are preserved through foreign key relationships. Re-joining all tables reconstructs original data without spurious tuples. The database satisfies 5NF.

9. Database Implementation

9.1 SQL Implementation

DDL Commands — Table Creation:

```
CREATE DATABASE agriculture_management;  
USE agriculture_management;
```

```
CREATE TABLE Farmers (  
    Farmer_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Name      VARCHAR(100) NOT NULL,  
    Contact   VARCHAR(100),  
    Location  VARCHAR(100)  
);
```

```
CREATE TABLE Fields (  
    Field_ID   INT AUTO_INCREMENT PRIMARY KEY,  
    Field_Name VARCHAR(100) NOT NULL,  
    Area_Acres DECIMAL(10,2),  
    Crop_Type  VARCHAR(100),  
    Farmer_ID  INT,  
    FOREIGN KEY (Farmer_ID) REFERENCES  
Farmers(Farmer_ID)  
);
```

```
CREATE TABLE SoilRecords (  
    Record_ID   INT AUTO_INCREMENT PRIMARY KEY,  
    Field_ID    INT,  
    Nitrogen     DECIMAL(5,2),  
    Phosphorus  DECIMAL(5,2),  
    Potassium    DECIMAL(5,2),  
    pH_Level     DECIMAL(4,2),  
    Recorded_At  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (Field_ID) REFERENCES Fields(Field_ID)  
);
```

```
CREATE TABLE FertilizerUsage (  
    Usage_ID      INT AUTO_INCREMENT PRIMARY KEY,  
    Field_ID      INT,  
    Fertilizer_Name VARCHAR(100),  
    Quantity_kg   DECIMAL(10,2),
```

```

        Applied_At      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (Field_ID) REFERENCES Fields(Field_ID)
    );

```

```

CREATE TABLE CropDiseases (
    Disease_ID    INT AUTO_INCREMENT PRIMARY KEY,
    Field_ID      INT,
    Disease_Name  VARCHAR(150),
    Severity      ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    Logged_At     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (Field_ID) REFERENCES Fields(Field_ID)
);

```

```

CREATE TABLE Alerts (
    Alert_ID      INT AUTO_INCREMENT PRIMARY KEY,
    Field_ID      INT,
    Alert_Message TEXT,
    Created_At    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (Field_ID) REFERENCES Fields(Field_ID)
);

```

```

CREATE TABLE AuditLog (
    Log_ID        INT AUTO_INCREMENT PRIMARY KEY,
    Field_ID      INT,
    Action        VARCHAR(100),
    Details       TEXT,
    Logged_At     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (Field_ID) REFERENCES Fields(Field_ID)
);

```

```

CREATE TABLE FertilizerSummary (
    Summary_ID    INT AUTO_INCREMENT PRIMARY KEY,
    Total_Quantity_kg DECIMAL(15,2),
    Last_Updated  TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
);

```

DML Commands — INSERT:

```

INSERT INTO Farmers VALUES
(1, 'Gurpreet Singh', '9876543210', 'Amritsar'),

```

```
(2, 'Manpreet Kaur', '9876543211', 'Ludhiana'),  
(3, 'Harjinder Singh', '9876543212', 'Patiala'),  
(4, 'Sukhjit Kaur', '9876543213', 'Bathinda');
```

```
INSERT INTO Fields VALUES  
(1, 'North Plot', 12.5, 'Wheat', 1),  
(2, 'South Field', 8.3, 'Rice', 2),  
(3, 'East Block', 15.0, 'Sugarcane', 3),  
(4, 'West Patch', 6.7, 'Cotton', 4);
```

DML Commands — UPDATE & DELETE:

```
UPDATE SoilRecords SET pH_Level = 7.0 WHERE Field_ID =  
2;  
  
DELETE FROM Alerts WHERE Created_At < DATE_SUB(NOW(),  
INTERVAL 30 DAY);
```

SELECT Queries — Joins:

```
-- All farmers with their fields  
SELECT f.Name, fd.Field_Name, fd.Crop_Type  
FROM Farmers f JOIN Fields fd ON f.Farmer_ID =  
fd.Farmer_ID;  
  
-- Disease list with field and severity  
SELECT fd.Field_Name, cd.Disease_Name, cd.Severity  
FROM CropDiseases cd JOIN Fields fd ON cd.Field_ID =  
fd.Field_ID  
ORDER BY cd.Severity DESC;
```

SELECT Queries — Subqueries:

```
-- Fields with above-average nitrogen  
SELECT Field_Name FROM Fields f  
WHERE Field_ID IN (  
    SELECT Field_ID FROM SoilRecords  
    WHERE Nitrogen > (SELECT AVG(Nitrogen) FROM  
SoilRecords)  
);
```

SELECT Queries — Aggregate Functions, GROUP BY, HAVING:

```
-- Total fertilizer per field, only fields using more
than 40 kg
SELECT fd.Field_Name, SUM(fu.Quantity_kg) AS Total_kg
FROM FertilizerUsage fu
JOIN Fields fd ON fu.Field_ID = fd.Field_ID
GROUP BY fd.Field_Name
HAVING SUM(fu.Quantity_kg) > 40;
```

Views:

```
CREATE VIEW FieldHealthSummary AS
SELECT fd.Field_Name, fd.Crop_Type,
       sr.Nitrogen, sr.Phosphorus, sr.Potassium,
       sr.pH_Level,
       COUNT(cd.Disease_ID) AS Disease_Count
FROM Fields fd
LEFT JOIN SoilRecords sr ON fd.Field_ID = sr.Field_ID
LEFT JOIN CropDiseases cd ON fd.Field_ID = cd.Field_ID
GROUP BY fd.Field_ID, fd.Field_Name, fd.Crop_Type,
         sr.Nitrogen, sr.Phosphorus, sr.Potassium,
         sr.pH_Level;
```

9.2 PL/SQL Components

Stored Procedure — GetFieldSummary:

Accepts a Field_ID and returns a complete summary: field info, latest soil nutrients, total fertilizer applied, and disease count. Offloads aggregation to the database for fast API response.

```
DELIMITER $$
CREATE PROCEDURE GetFieldSummary(IN p_field_id INT)
BEGIN
    SELECT f.Field_ID, f.Field_Name, f.Crop_Type,
    f.Area_Acres,
           sr.Nitrogen, sr.Phosphorus, sr.Potassium,
    sr.pH_Level,
           COALESCE(SUM(fu.Quantity_kg), 0) AS
    Total_Fertilizer_kg,
           COUNT(DISTINCT cd.Disease_ID) AS
    Disease_Count
    FROM Fields f
    LEFT JOIN SoilRecords sr ON f.Field_ID = sr.Field_ID
    LEFT JOIN FertilizerUsage fu ON f.Field_ID =
    fu.Field_ID
    LEFT JOIN CropDiseases cd ON f.Field_ID =
    cd.Field_ID
    WHERE f.Field_ID = p_field_id
    GROUP BY f.Field_ID, f.Field_Name, f.Crop_Type,
           f.Area_Acres, sr.Nitrogen, sr.Phosphorus,
           sr.Potassium, sr.pH_Level;
END$$
DELIMITER ;
```

Stored Function — GetTotalFertilizerForField:

Returns the total fertilizer (in kg) applied to a specific field. Used in reports and dashboards.

```
DELIMITER $$
CREATE FUNCTION GetTotalFertilizerForField(p_field_id
INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE total DECIMAL(10,2);
```



```

        SELECT COALESCE(SUM(Quantity_kg),0) INTO total
        FROM FertilizerUsage WHERE Field_ID = p_field_id;
        RETURN total;

    END$$
DELIMITER ;

```

Trigger 1 — ValidateFertilizerData (BEFORE INSERT):

Prevents fertilizer entries exceeding 100 kg per application — enforces data integrity at the database level before any record is committed.

```

DELIMITER $$
CREATE TRIGGER ValidateFertilizerData
BEFORE INSERT ON FertilizerUsage
FOR EACH ROW
BEGIN
    IF NEW.Quantity_kg > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Fertilizer quantity exceeds
100 kg limit';
    END IF;
END$$
DELIMITER ;

```

Trigger 2 — HighSeverityDiseaseAlert (AFTER INSERT):

Fires after a crop disease record is inserted. If severity is HIGH or CRITICAL, it automatically inserts an alert message into the Alerts table.

```

DELIMITER $$
CREATE TRIGGER HighSeverityDiseaseAlert
AFTER INSERT ON CropDiseases
FOR EACH ROW
BEGIN
    IF NEW.Severity IN ('HIGH','CRITICAL') THEN
        INSERT INTO Alerts (Field_ID, Alert_Message)
        VALUES (NEW.Field_ID,
            CONCAT('ALERT: ',NEW.Severity,' severity disease
[' ,
                NEW.Disease_Name,'] on Field
#',NEW.Field_ID,
                '. Immediate action required.'));
    END IF;
END$$

```

```
DELIMITER ;
```

Trigger 3 — LogFertilizerUsage (AFTER INSERT):

Fires after every fertilizer insert. Records the action type, fertilizer name, quantity, and field reference in the immutable AuditLog table.

```
DELIMITER $$
CREATE TRIGGER LogFertilizerUsage
AFTER INSERT ON FertilizerUsage
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (Field_ID, Action, Details)
    VALUES (NEW.Field_ID, 'FERTILIZER_APPLIED',
            CONCAT('Applied ', NEW.Quantity_kg, ' kg of ',
                NEW.Fertilizer_Name, ' on Field
#', NEW.Field_ID));
END$$
DELIMITER ;
```

Cursor — ProcessFieldAlerts:

Iterates over all unresolved HIGH/CRITICAL disease records for a given field, printing a summary. Demonstrates row-by-row cursor processing with exception handling.

```
DELIMITER $$
CREATE PROCEDURE ProcessFieldAlerts(IN p_field_id INT)
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_disease VARCHAR(150);
    DECLARE v_severity
ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL');
    DECLARE cur CURSOR FOR
        SELECT Disease_Name, Severity FROM CropDiseases
        WHERE Field_ID = p_field_id
        AND Severity IN ('HIGH', 'CRITICAL');
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_disease, v_severity;
        IF done THEN LEAVE read_loop; END IF;
        SELECT CONCAT('[', v_severity, '] ', v_disease) AS
Alert_Summary;
    END LOOP;
END
```

```
END LOOP;  
CLOSE cur;  
END$$  
DELIMITER ;
```

MySQL Event Scheduler — RecalculateFertilizerSummary:

Runs every 24 hours. Recalculates global total fertilizer applied across all fields and caches it in FertilizerSummary for fast dashboard reads.

```
SET GLOBAL event_scheduler = ON;  
  
DELIMITER $$  
CREATE EVENT RecalculateFertilizerSummary  
ON SCHEDULE EVERY 1 DAY  
STARTS CURRENT_TIMESTAMP  
DO  
BEGIN  
    INSERT INTO FertilizerSummary (Total_Quantity_kg)  
    SELECT COALESCE(SUM(Quantity_kg),0) FROM  
FertilizerUsage;  
END$$  
DELIMITER ;
```

10. Transaction Management & Concurrency

Use of Transactions

The system uses explicit transaction blocks for multi-step operations to ensure atomicity — either all steps succeed or all are rolled back.

```
START TRANSACTION;

-- Step 1: Insert new field
INSERT INTO Fields (Field_Name,Area_Acres,Crop_Type,Farmer_ID)
VALUES ('New Plot',10.0,'Maize',1);

-- Step 2: Insert initial soil record
INSERT INTO SoilRecords (Field_ID,Nitrogen,Phosphorus,Potassium,pH_Level)
VALUES (LAST_INSERT_ID(),40.0,18.0,25.0,6.9);

SAVEPOINT after_soil;

-- Step 3: Insert fertilizer (may fail validation trigger)
INSERT INTO FertilizerUsage (Field_ID,Fertilizer_Name,Quantity_kg)
VALUES (LAST_INSERT_ID(),'Urea',45.0);

-- If trigger raises error:
-- ROLLBACK TO SAVEPOINT after_soil;

COMMIT;
```

ACID Properties

- Atomicity — transactions ensure all inserts succeed together or roll back entirely.
- Consistency — foreign key constraints and trigger validations maintain referential and domain integrity.
- Isolation — MySQL's default REPEATABLE READ isolation prevents dirty reads during concurrent field summary queries.
- Durability — committed transactions are persisted to disk via MySQL's WAL (Write-Ahead Logging) through InnoDB's redo log.

Concurrency Control

MySQL InnoDB uses row-level locking for INSERT/UPDATE/DELETE operations, minimizing contention between concurrent users. The stored procedure GetFieldSummary uses consistent reads (snapshot isolation under REPEATABLE READ) to return accurate aggregated data without blocking concurrent writes.

11. Tools & Technologies Used

Category	Technology / Tool
DBMS	MySQL 8.0+
Query Language	SQL / PL-SQL style (Stored Procedures, Functions, Triggers, Cursors, Events)
Backend	Node.js, Express.js, mysql2 (promise pool)
Frontend	React, Vite, Tailwind CSS v4, Recharts, Framer Motion, Axios
DB Interface Tool	MySQL Workbench 8.0
Version Control	Git, GitHub (github.com/brarprabh/agriculture-management-system)

12. Actual Outcomes

The following outcomes were successfully achieved upon full implementation of the Agriculture Management System database:

- Successfully created a fully normalized relational database with 8 tables, normalized up to Fifth Normal Form (5NF), with all primary keys, foreign keys, and referential integrity constraints correctly enforced by MySQL.
- Efficient data retrieval implemented via 8 SQL SELECT queries covering inner joins, left joins, subqueries, aggregate functions (SUM, COUNT, AVG), GROUP BY, HAVING clauses, and a reusable view (FieldHealthSummary).
- Complete automation using 3 triggers: ValidateFertilizerData (BEFORE INSERT validation), HighSeverityDiseaseAlert (AFTER INSERT auto-alerting), and LogFertilizerUsage (AFTER INSERT audit logging) — all firing at the database engine layer with zero application code.
- Stored procedure GetFieldSummary successfully returns a complete per-field analytical summary (soil nutrients, total fertilizer, disease count) in a single database call, reducing API response complexity.
- Stored function GetTotalFertilizerForField correctly computes cumulative fertilizer applied per field, usable inline in SELECT queries.
- Cursor-based procedure ProcessFieldAlerts successfully iterates over HIGH/CRITICAL disease records row-by-row, demonstrating explicit cursor lifecycle (DECLARE, OPEN, FETCH, CLOSE) with NOT FOUND exception handling.
- MySQL Event Scheduler (RecalculateFertilizerSummary) verified to run every 24 hours, correctly recalculating and caching global fertilizer totals in the FertilizerSummary table.
- Transaction management validated — multi-step field registration (field + soil record + fertilizer) rolls back atomically on trigger validation failure, with SAVEPOINT allowing partial rollback without losing the field creation.
- Improved data consistency and integrity confirmed — all invalid inserts (pH out of 0–14 range, fertilizer > 100 kg) are blocked at the database level before reaching application logic.

Overall, the project demonstrates that a well-designed relational database with proper normalization, constraint enforcement, and PL/SQL-style automation can eliminate the inefficiencies of manual agricultural record-keeping and provide a scalable foundation for future enhancements such as IoT sensor integration, predictive disease modelling, and mobile-based farmer dashboards.